Black Cat Nails Móvil: React Native y Figma Aplicados al Diseño e Implementación de la Aplicación para Clientas de un Emprendimiento de Manicura

Enrique Alejandro García Villeda (GV200136)

Facultad de Ingeniería, Universidad Don Bosco

DPS941: Diseño y Programación de Software Multiplataforma, G01T

Alexander Alberto Siguenza Campos

22 de octubre de 2026

Introducción

Este documento es el Avance 1 de la Etapa 3 del proyecto de cátedra de DPS941, entregado como Foro de Discusión 1. En la Etapa 1 describí el problema de Black Cat Nails by Vante: la propietaria pasa entre 1.6 y 2.3 veces más tiempo coordinando citas por mensajería que atendiendo clientas, y las inasistencias rondan entre el 12 % y el 19 % sin que tenga cómo evitarlas (García Villeda, 2026). En la Etapa 2 construí el panel web administrativo con React y Next.js. Con esta entrega empieza la aplicación móvil, que es la que usa la clienta para agendar y revisar sus citas por su cuenta.

El objetivo del avance es llevar a React Native las pantallas principales del prototipo de Figma, con navegación, inicio de sesión y validación de formularios. Además de describir las herramientas, explico qué características de React Native y qué partes del trabajo en Figma influyeron en las decisiones de la aplicación, y anoto dónde me aparté del prototipo y por qué.

Primero presento los conceptos de React Native y de Figma que usé. Después documento el proceso de diseño UX/UI. Al final describo la implementación, las pruebas y la comparación entre el prototipo y la aplicación.

React Native en el Proyecto

Qué es React Native y Por Qué se Eligió

React Native es un framework de código abierto creado por Meta para construir aplicaciones de Android e iOS con JavaScript o TypeScript y el modelo de componentes de React. Las soluciones híbridas muestran una página web dentro de un contenedor; React Native, en cambio, convierte cada componente en una vista nativa: un `<View>` termina siendo un `android.view.View` en Android y un `UIView` en iOS (Meta Platforms, s.f.-b). Román Gallardo et al. (s.f.) lo resumen así: una sola base de código, con la apariencia y el rendimiento de una app nativa.

Lo elegí por dos razones. La clienta ya contacta al negocio desde el teléfono, por Instagram y mensajería, así que tenía sentido que la aplicación estuviera ahí. Y soy el único desarrollador del proyecto: con React Native puedo usar el mismo lenguaje, el mismo manejo de estado y hasta los mismos módulos de lógica de negocio que ya funcionan en la versión web.

Arquitectura de React Native

Desde la versión 0.76, React Native trae activada por defecto la Nueva Arquitectura. Tiene tres piezas principales: JSI (JavaScript Interface), que deja a JavaScript llamar directamente a código nativo sin pasar mensajes serializados por un puente asíncrono; Fabric, el nuevo sistema de renderizado; y los Turbo Modules, que cargan los módulos nativos solo cuando hacen falta (Meta Platforms, s.f.-c). El JavaScript corre en Hermes, un motor pensado para móviles que compila el código a bytecode antes de ejecutarlo.

En este proyecto eso se nota en la fluidez de la interfaz. Cada vez que la clienta elige un día, la pantalla de fecha recalcula hasta veinte bloques horarios, y la galería es una cuadrícula de imágenes que se desplaza. Si la comunicación entre JavaScript y la parte nativa fuera lenta, ambas pantallas se sentirían trabadas.

Expo como Entorno de Desarrollo

Creé el proyecto con Expo SDK 58, que trae React Native 0.88 y React 19. Expo es un framework sobre React Native que agrega herramientas de desarrollo, módulos nativos

mantenidos por su equipo (cámara, galería, portapapeles, criptografía, fuentes) y la app Expo Go, con la que se abre el proyecto en un teléfono real escaneando un código QR, sin compilar en Android Studio ni en Xcode (Expo, s.f.). Román Gallardo et al. (s.f.) dedican el segundo capítulo de su libro a este flujo de trabajo, justamente porque facilita empezar.

Me puse una condición al usar Expo: cada dependencia tenía que estar incluida en Expo Go, para que la propietaria pudiera probar la app en su teléfono sin instalar una compilación especial. Por eso usé `expo-image-picker` para las imágenes, `expo-clipboard` para copiar el código de referido, `expo-crypto` para el hash de las contraseñas y `@react-native-async-storage/async-storage` para guardar los datos en el teléfono. Instalé todo con `npx expo install`, que elige la versión compatible con el SDK.

Componentes, Props y Estado

En lugar de etiquetas HTML, React Native usa componentes núcleo: `View` como contenedor, `Text` para todo texto, `Image`, `TextInput`, `ScrollView`, `FlatList` y `Pressable` (Meta Platforms, s.f.-b). En la Tabla 1 indico dónde usé cada uno y por qué.

Tabla 1

Componentes núcleo de React Native usados en la aplicación

Componente	Uso en Black Cat Nails	Razón de la elección
FlatList	Galería de diseños (2 columnas) y carrusel de destacados	Virtualiza la lista: solo monta las tarjetas visibles. La galería crecerá con cada trabajo que publique la propietaria.
ScrollView	Inicio, formularios, perfil, filtros horizontales	Contenido acotado y heterogéneo, donde la virtualización no aporta.
Pressable	Botones, tarjetas, días del calendario, bloques horarios	Expone el estado `pressed` para dar respuesta visual y acepta `accessibilityState` (deshabilitado, seleccionado).
TextInput	Login, registro, notas de la cita, edición de perfil	Permite elegir el teclado (`email-address`, `number-pad`) y el autocompletado del sistema.
KeyboardAvoidingView	Login y registro	En iOS el teclado tapa el botón de envío; en Android el sistema ya ajusta la ventana, por eso el comportamiento se activa solo en iOS.
SafeAreaView	Contenedor de todas las pantallas	Evita que el contenido quede debajo de la muesca o de la barra de gestos.

Nota. Elaboración propia con base en Meta Platforms (s.f.-b) y en el código del proyecto.

Los componentes reciben datos por props y manejan su propio estado con hooks. Román Gallardo et al. (s.f.) describen las props como la forma en que un componente padre configura a un hijo sin que el hijo sepa de dónde salen los datos. En el proyecto, `TarjetaCita` recibe una cita y, si hace falta, sus botones de acción. Así la misma tarjeta muestra “Reprogramar” y “Cancelar” a la clienta, y “Confirmar”, “Iniciar” o “Completar” a la administradora, sin tocar su código.

Dividí el estado global en tres providers de Context API: `AuthContext` (sesión y rol), `CatalogoContext` (servicios, diseños y favoritos) y `AgendaContext` (citas y borrador del agendamiento). En la Etapa 1 justifiqué esa separación para que un cambio en una parte no vuelva a dibujar pantallas de otra (García Villeda, 2026). En el móvil se ve al marcar un diseño como favorito: se actualiza la galería, pero la lista de citas no se vuelve a renderizar.

Estilos y Flexbox

React Native no usa CSS. Los estilos son objetos de JavaScript creados con `StyleSheet.create`, y el layout lo calcula Yoga, una implementación de Flexbox. Dos diferencias con la web me afectaron. La dirección por defecto es columna, no fila. Y no hay herencia en cascada: un `Text` no toma el color de texto del `View` que lo contiene, así que cada uno lo declara (Meta Platforms, s.f.-b). Por esa segunda razón junté colores y tipografías en un archivo de tokens, que describo en la sección de implementación.

Navegación con React Navigation

React Native no incluye un sistema de rutas, así que usé React Navigation 7 con dos navegadores: *native stack*, que usa las transiciones propias de cada plataforma, y *bottom tabs* para la barra inferior (React Navigation, s.f.). Consideré Expo Router, pero lo descarté: el enrutamiento por archivos ayuda poco en una app donde lo que cambia el flujo es el rol de la usuaria, no una dirección web.

Lo más importante de la navegación es que depende del rol. El navegador raíz no esconde la pantalla de administración a la clienta; simplemente no la monta. Sin sesión solo existen Login y Registro. Con sesión de clienta aparecen sus cuatro pestañas y el flujo de agendamiento. Con sesión de administradora, la agenda y su perfil. De esta forma la clienta no puede llegar a la vista de la administradora, ni siquiera si un botón quedara mal programado.

Figma en el Proceso de Diseño

Figma como Herramienta de Diseño de Interfaces

Figma es una herramienta de diseño de interfaces que corre en el navegador, y diseño y desarrollo trabajan sobre el mismo archivo. Usé cuatro de sus funciones: *frames* del tamaño de un teléfono (390 × 844 px, lo que mide un teléfono de gama media actual), variables de color, Auto Layout y la inspección de propiedades para desarrollo (Figma, s.f.-a, s.f.-b, s.f.-c).

Variables y Sistema de Diseño

Antes de dibujar pantallas definí cinco variables de color a partir del logotipo del negocio, que sus clientas ya reconocen por Instagram: Dark, Berry Crush, Petal Pink, Wisteria Blue y White. Una variable de Figma es un valor con nombre que se aplica a muchos elementos; si cambia, cambian todos (Figma, s.f.-b). Esto se trasladó directo al código: usé los mismos nombres en `src/theme/tokens.ts`, así que si un color cambia en Figma sé exactamente qué línea editar. Para títulos elegí Playfair Display, que tiene un aire editorial que encaja con el rubro de belleza, y para textos y formularios Inter, que se lee bien en tamaños pequeños.

Auto Layout y su Correspondencia con Flexbox

Auto Layout es la función de Figma que acomoda los elementos de un frame según reglas de dirección, espaciado y relleno, en vez de coordenadas fijas (Figma, s.f.-a). Funciona casi igual que Flexbox, por eso un prototipo hecho con Auto Layout se pasa a React Native sin mucha interpretación. La Tabla 2 muestra la equivalencia que seguí.

Tabla 2

Correspondencia entre propiedades de Auto Layout en Figma y estilos de React Native

Figma (Auto Layout)	React Native	Ejemplo en el proyecto
Dirección vertical / horizontal	`flexDirection: 'column' | 'row'`	Fila de servicio: miniatura, textos y flecha en `row`
Espaciado entre elementos	`gap`	16 px entre campos del login (`gap: 16`)
Relleno (padding)	`padding`, `paddingHorizontal`	Login con 64 px arriba y 32 px a los lados
Alineación	`alignItems`, `justifyContent`	Encabezado del login centrado
Fill container	`flex: 1` o `alignSelf: 'stretch'`	Botones “Reprogramar” y “Cancelar” que se reparten el ancho
Space between	`justifyContent: 'space-between'`	Login: logotipo arriba, formulario al centro, enlace de registro abajo
Radio de esquina	`borderRadius`	16 px en tarjetas, botones y campos

Nota. Elaboración propia. Los valores se leyeron del archivo de Figma del proyecto con el modo de inspección.

Estados de Componente y Flujos

Si un prototipo solo muestra el caso en que todo sale bien, el desarrollador termina adivinando qué pasa cuando algo falla. Por eso diseñé el login en tres frames (*login-default*, *login-error-state* y *login-loading-state*) y “Mis citas” en otros tres: próximas, historial y vacío. Cada frame lleva un nombre que dice su estado, y ese mismo nombre aparece en un comentario al inicio de la pantalla en el código. Así cualquier pantalla se puede rastrear hasta su diseño.

Proceso de Diseño UX/UI Aplicado

Del Problema a las Decisiones de Interfaz

Arranqué con una pregunta: ¿qué necesita poder hacer la clienta sola para que la propietaria deje de contestar mensajes todo el día? La respuesta fueron cuatro tareas: ver su próxima cita, explorar diseños, agendar con la seguridad de que el horario está libre y administrar sus citas. Después revisé cada pantalla con las heurísticas de usabilidad de Nielsen (1994). La Tabla 3 resume las decisiones principales.

Tabla 3

Decisiones de diseño y heurística de usabilidad que las sustenta

Decisión	Pantalla	Heurística	Justificación en el negocio
Próxima cita como primera tarjeta	Inicio	Visibilidad del estado del sistema	"¿A qué hora era mi cita?" es la consulta más repetida por mensajería.
Horarios que no caben aparecen tachados	Paso 2: Fecha y hora	Prevención de errores	El cuaderno ya produjo citas encimadas. Mejor impedir la elección que rechazarla después.
Total de tiempo y dinero siempre visible	Pasos 1 y 2	Visibilidad del estado del sistema	La clienta combina servicios y necesita saber cuánto dura y cuesta antes de elegir hora.
Indicador "Paso 1 · 2 · 3"	Agendamiento	Control y libertad del usuario	Muestra cuánto falta y permite volver sin perder lo elegido.
Política de cancelación explícita y casilla de asistencia	Paso 3: Confirmación	Prevención de errores	La propietaria descartó el anticipo; el compromiso explícito es la medida contra el ausentismo.
Error debajo del campo y en rojo	Login y registro	Ayudar a reconocer y corregir errores	La clienta ve qué campo falla y por qué, sin un mensaje genérico.
Estado vacío con acción "Agendar cita"	Mis citas	Reconocer antes que recordar	Una clienta nueva no queda frente a una pantalla en blanco.
Aviso "Es hora de tu retoque"	Inicio e historial	Visibilidad del estado del sistema	Aprovecha el ciclo de retorno mensual para subir de 4 a 10 clientas semanales.

Nota. Elaboración propia con base en Nielsen (1994) y en el diagnóstico de la Etapa 1.

Sistema de Diseño y Accesibilidad

Revisé los colores con la razón de contraste de las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2, que piden al menos 4.5:1 para texto normal (W3C, 2023). Los resultados están en la Tabla 4. La única combinación que falla es texto blanco sobre Petal Pink, que el prototipo usaba en el botón del login. En la app todos los botones primarios llevan texto

oscuro, como ya tenían “Continuar” y “Confirmar cita” en el mismo prototipo, así que de paso quedaron todos iguales.

Tabla 4

Tokens de color y razón de contraste según WCAG 2.2

Token (Figma)	Valor	Uso	Combinación	Contraste
Dark	#191308	Fondo principal	Texto blanco	18.45:1 (cumple)
Berry Crush	#B5446E	Tarjeta de próxima cita, resumen	Texto blanco	5.23:1 (cumple)
Petal Pink	#E477C1	Botones y elementos activos	Texto Dark	6.77:1 (cumple)
Petal Pink	#E477C1	Botón del login en Figma	Texto blanco	2.72:1 (no cumple)
Wisteria Blue	#9CA3DB	Bordes, enlaces, iconos	Sobre Dark	7.62:1 (cumple)

Nota. Elaboración propia. Contraste calculado con la fórmula de luminancia relativa de WCAG 2.2 (W3C, 2023).

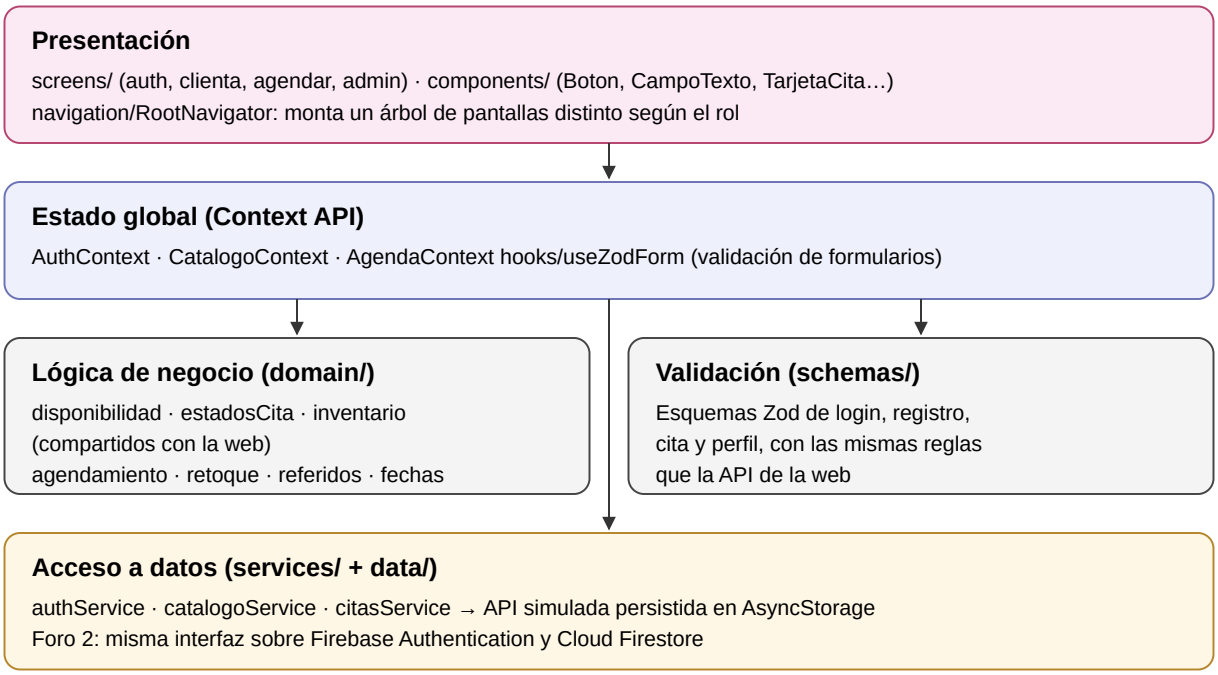
Implementación en React Native

Arquitectura por Capas

La aplicación sigue la arquitectura de tres capas que definí en la Etapa 1. En la Figura 1 se ve el recorrido de una acción, por ejemplo confirmar una cita, desde la pantalla hasta los datos.

Figura 1

Arquitectura por capas de la aplicación móvil



Nota. Elaboración propia (2026). Las pantallas solo conocen los contextos, y los contextos solo conocen los servicios. Cambiar la API simulada por Firebase no toca las capas superiores.

La decisión que más trabajo me ahorró fue copiar a la app móvil, sin cambiarles nada, los módulos `disponibilidad.ts`, `estadosCita.ts` e `inventario.ts` de la versión web. Son funciones puras de TypeScript, sin dependencias de React, de Next.js ni de la base de datos. Funcionan igual en el servidor de Vercel que en el teléfono, y una cita válida en la web también

lo es en el móvil. Este fue uno de los argumentos con los que elegí el ecosistema React en la Etapa 1, y aquí se comprobó.

De los Tokens de Figma al Código

La Figura 2 muestra el archivo de tokens. Todas las pantallas toman de ahí sus colores, tipografías y radios; en los componentes no hay colores escritos a mano.

Figura 2

Fragmento de src/theme/tokens.ts con las variables de Figma

```
export const colors = {
  // Variables de Figma
  dark: '#191308',          // fondo principal (bg-base)
  berryCrush: '#B5446E',    // tarjetas destacadas y encabezados (surface)
  petalPink: '#E477C1',     // botones principales y elementos activos (primary)
  wisteriaBlue: '#9CA3DB',  // estados, etiquetas e iconografía (accent)
  white: '#FFFFFF',
  ...
} as const;

export const fonts = {
  display: 'PlayfairDisplay_700Bold',
  regular: 'Inter_400Regular',
  semibold: 'Inter_600SemiBold',
  ...
} as const;
```

Nota. Elaboración propia. Las fuentes se cargan con `expo-font` desde los paquetes `@expo-google-fonts` antes de mostrar la primera pantalla.

Trazabilidad entre el Prototipo y las Pantallas

En la Tabla 5 relaciono los quince frames móviles del archivo de Figma con la pantalla que los implementa.

Tabla 5

Frames de Figma y pantallas de React Native que los implementan

Frame en Figma	Pantalla	Comportamiento implementado
login-default, login-error-state, login-loading-state	LoginScreen	Validación al salir del campo y al enviar, mensaje de credenciales incorrectas, botón con indicador de carga
home-catalogo	InicioScreen	Próxima cita real, aviso de retoque, servicios que abren el agendamiento con el servicio elegido
Gallery Grid Screen	GaleriaScreen	Filtros por categoría, búsqueda, favoritos persistentes por usuaria
Detail Modal Screen	DisenoDetalleScreen	Hoja inferior sobre la galería; “Agendar este diseño” preselecciona el servicio sugerido
screen-4a-servicios	AgendarServiciosScreen	Selección múltiple con total de duración y monto
screen-4b-fecha	AgendarFechaScreen	Calendario con días cerrados o pasados deshabilitados; horarios calculados
screen-4c-confirmacion	AgendarConfirmacionScreen	Resumen, notas validadas, imagen de referencia, aceptación de la política
screen-4d-exito	AgendarExitoScreen	Comprobante con fecha, hora y total
m5-mis-citas-proximas, m5-mis-citas-historial, m5-mis-citas-empty	MisCitasScreen	Pestañas, estados con color, cancelación con regla de 12 horas, aviso de retoque, estado vacío
m6-perfil-referidos	PerfilScreen	Foto de perfil, edición validada de correo y teléfono, copiar y compartir código, conteo de referidas

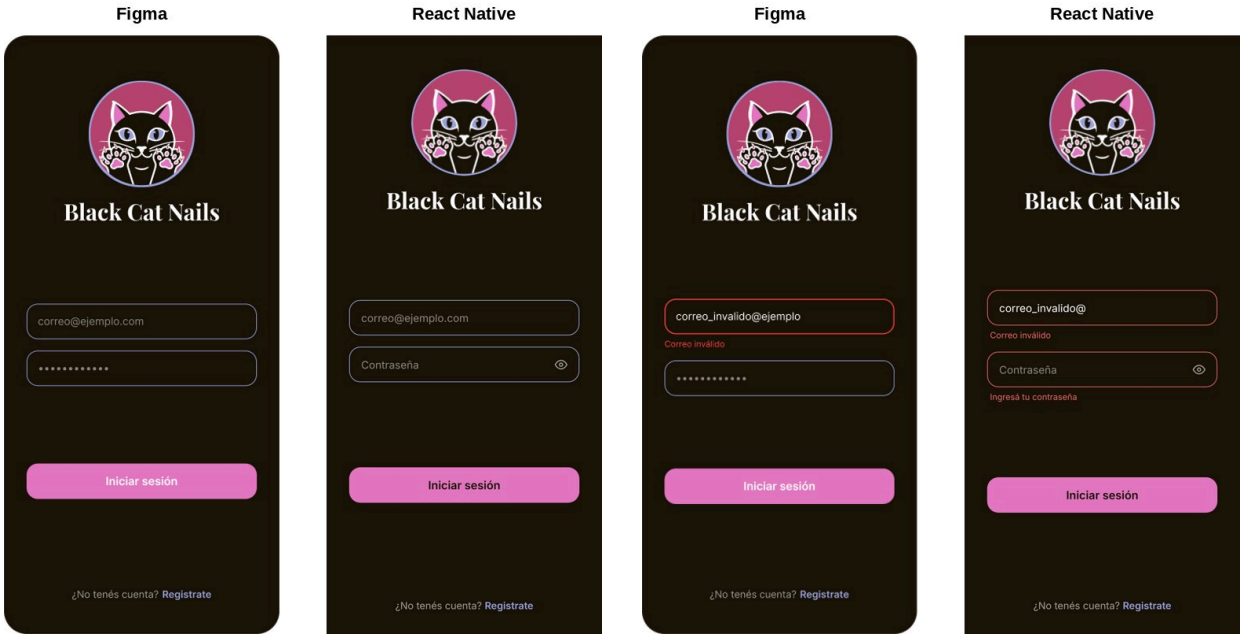
Nota. Elaboración propia. Registro y las dos vistas de la administradora no tenían frame móvil; se construyeron con los mismos componentes y tokens.

Comparación Visual entre Prototipo e Implementación

De la Figura 3 a la 7 comparo cada frame de Figma con una captura de la aplicación funcionando. Tomé las capturas con la misma resolución lógica del prototipo (390 × 844).

Figura 3

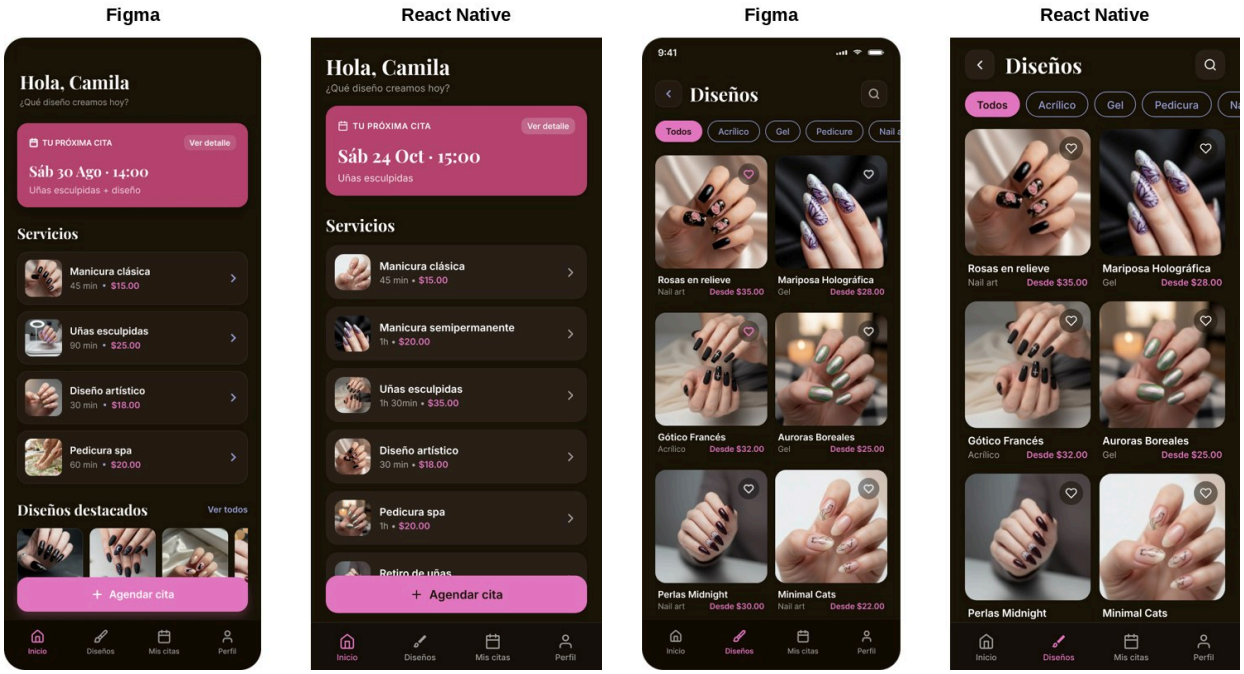
Pantalla de inicio de sesión en estado normal y con error de validación



Nota. A la izquierda de cada par, el frame de Figma; a la derecha, la aplicación. La implementación agrega el icono para mostrar u ocultar la contraseña y valida también el campo vacío. Elaboración propia mediante Figma y React Native (2026).

Figura 4

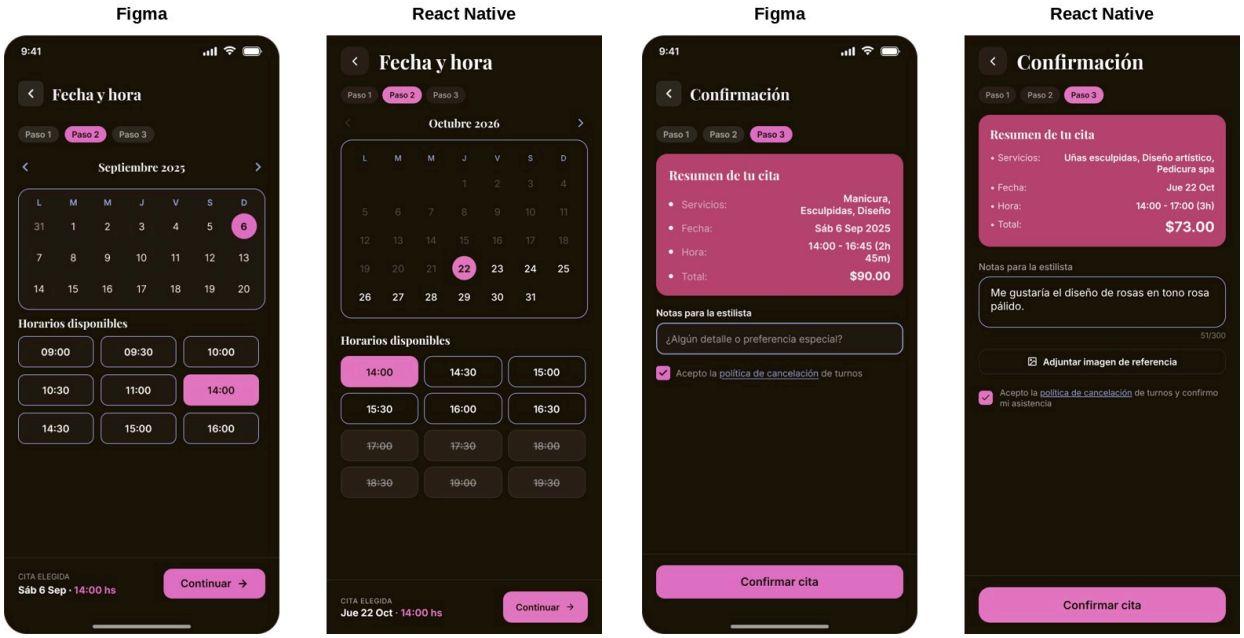
Pantalla de inicio y galería de diseños



Nota. Los precios y duraciones provienen del catálogo real del negocio cargado en la versión web, por eso difieren de los valores de ejemplo del prototipo. La categoría “Pedicure” del prototipo se unificó como “Pedicura”, que es como se llama en el catálogo. Elaboración propia (2026).

Figura 5

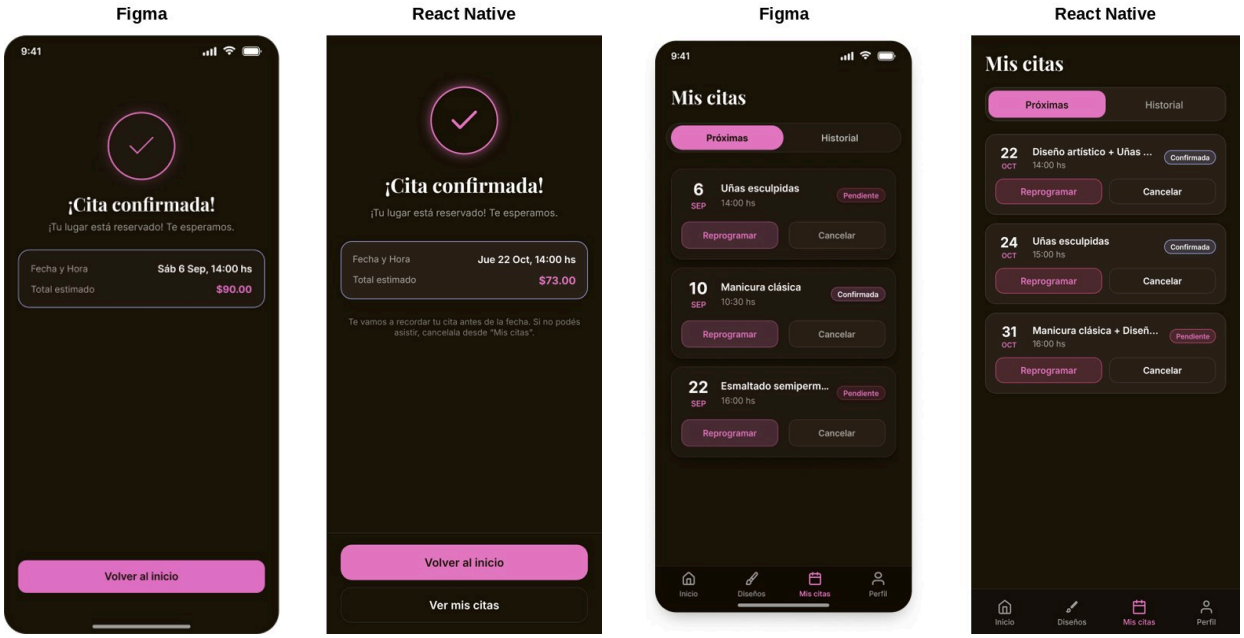
Pasos 2 y 3 del agendamiento: fecha y hora, y confirmación



Nota. Los bloques tachados no admiten la duración elegida (3 horas más 15 minutos de preparación) antes del cierre de las 20:00. En la confirmación se agregó el contador de caracteres de las notas y el botón para adjuntar una imagen de referencia, previsto en el módulo de galería de la Etapa 1. Elaboración propia (2026).

Figura 6

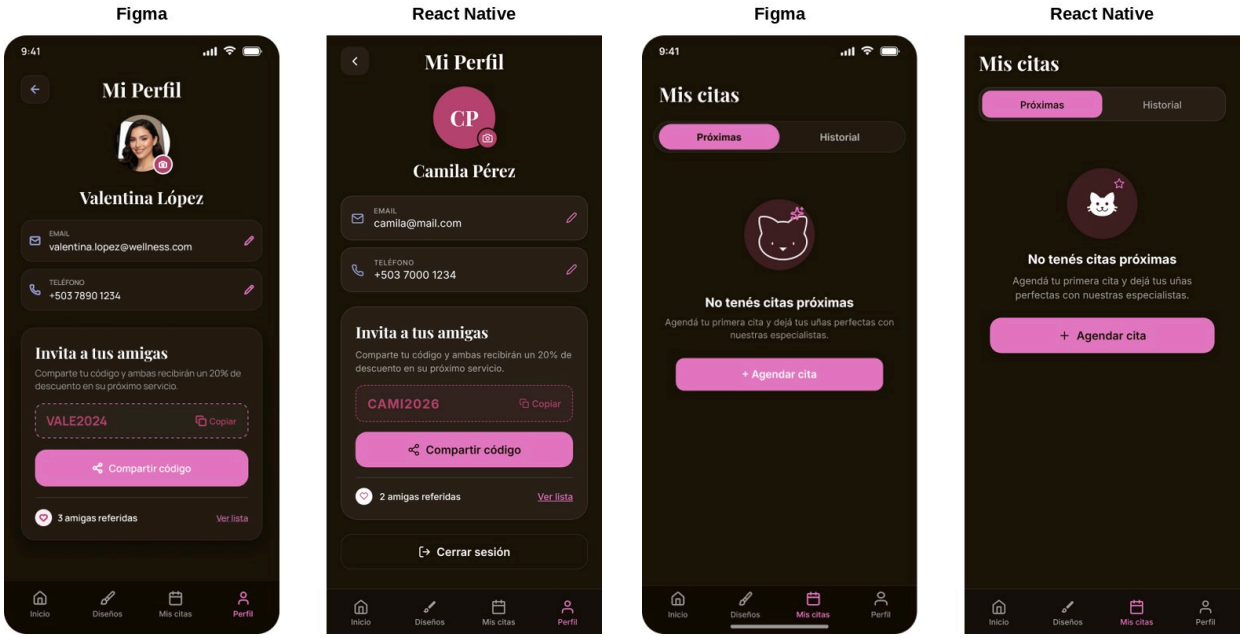
Comprobante de cita y lista de próximas citas



Nota. “Pendiente” corresponde al estado *solicitada* y “Confirmada” al estado *confirmada* de la máquina de estados. Elaboración propia (2026).

Figura 7

Perfil con módulo de referidos y estado vacío de Mis citas



Nota. Sin foto de perfil se muestran las iniciales; al tocar la cámara se abre la galería del teléfono con `expo-image-picker`. El estado vacío corresponde a una clienta recién registrada. Elaboración propia (2026).

Navegación y Control de Acceso por Rol

La Figura 8 muestra el centro del navegador raíz: una condición decide qué grupo de pantallas existe. La capa de datos, además, vuelve a revisar el rol en cada operación. `listarTodas` rechaza a una clienta, y `cambiarEstado` no deja que una clienta modifique la cita de otra. Es la misma doble protección de la web, donde el proxy de rutas protege la interfaz y cada endpoint revisa el rol por su lado.

Figura 8

Navegación condicional por sesión y rol en src/navigation/RootNavigator.tsx

```
export function RootNavigator() {
  const { usuaria, restaurando } = useAuth();

  if (restaurando) return <EstadoCargando texto="Abriendo Black Cat Nails…" />;

  return (
    <NavigationContainer theme={tema}>
      {!usuaria ? <NavegacionAuth />
        : usuaria.rol === 'admin' ? <NavegacionAdmin />
        : <NavegacionClienta />}
    </NavigationContainer>
  );
}
```

Nota. Elaboración propia. Al iniciar sesión no se navega manualmente: cambiar `usuaria` en el contexto basta para que React monte el árbol correcto.

Validación de Formularios

Escribí las reglas de validación como esquemas de Zod, una biblioteca que define la forma que deben tener los datos y devuelve los errores campo por campo (McDonnell, s.f.). Las reglas del registro son las mismas de la web, así que una cuenta que acepta una plataforma la acepta la otra. Para usar los esquemas en las pantallas escribí el hook `useZodForm`. Valida cada campo cuando la usuaria sale de él, vuelve a validar mientras corrige un campo con error y solo envía cuando todo el formulario está bien. Los servicios validan otra vez con el mismo esquema antes de guardar. Las reglas están en la Tabla 6.

Tabla 6

Reglas de validación por formulario

Formulario	Campo	Regla y mensaje
Login	Correo	Obligatorio y con formato de correo: “Correo inválido”
	Contraseña	Obligatoria. Credenciales incorrectas muestran un único mensaje, sin revelar cuál de los dos datos falló
Registro	Nombre	De 3 a 60 caracteres, solo letras y espacios
	Correo	Formato válido y no registrado previamente
	Teléfono	Exactamente 8 dígitos; el teclado es numérico y se descartan caracteres no numéricos
	Contraseña	Mínimo 8 caracteres, con al menos una letra y un número
	Confirmación	Debe coincidir con la contraseña
	Código de referido	Opcional; formato NOMBRE2026 y debe existir
Confirmación de cita	Notas	Máximo 300 caracteres, con contador visible
	Política	Debe aceptarse: “Debés aceptar la política de cancelación”
Perfil	Correo y teléfono	Mismas reglas del registro al editar

Nota. Elaboración propia. Esquemas en `src/schemas/`.

Lógica de Negocio del Agendamiento

La disponibilidad no se guarda en ningún lado; se calcula cada vez. Cuando la clienta elige servicios, `resumirSeleccion` suma las duraciones y `calcularDuracionTotal` agrega 15 minutos de preparación. Al elegir el día, `calcularHorarios` genera bloques cada 30 minutos dentro del horario (de 14:00 a 20:00 entre semana y de 9:00 a 17:00 los fines de semana) y bloquea los que terminarían después del cierre, los que ya pasaron y los que chocan con una cita solicitada, confirmada o en curso de cualquier clienta. En la Figura 5 se eligieron uñas esculpidas, diseño artístico y pedicura spa (3 horas más 15 minutos), y el último inicio posible queda a las 16:30. Al confirmar, el servicio calcula la disponibilidad de nuevo, porque mientras esta clienta elegía otra pudo haber tomado el mismo bloque.

Cada cita guarda una copia del nombre, el precio y la duración de sus servicios. Es la desnormalización que planteé en la Etapa 1: si la propietaria sube un precio, el historial sigue mostrando lo que se cobró. Cuando la administradora completa una cita, el sistema descuenta los insumos usados, registra el monto y calcula la fecha de retoque con el ciclo de retorno más largo entre los servicios. A partir de esa fecha la clienta ve el aviso “Es hora de tu retoque” en el inicio y en su historial.

Ajustes Respecto al Prototipo

Al implementar tuve que resolver cosas que el prototipo no definía:

- Texto oscuro en todos los botones primarios, por contraste (Tabla 4).
- Pantalla de registro y vistas de la administradora, que no tenían frame móvil, construidas con los mismos componentes.
- Bloques horarios no disponibles visibles y tachados, en vez de ocultos, para que la clienta entienda por qué no puede elegirlos.
- Botón para mostrar la contraseña y mensaje de credenciales incorrectas sobre el formulario.
- “Reprogramar” lleva al agendamiento con los mismos servicios ya elegidos; la cancelación de la cita original se hace aparte para no liberar el horario antes de tener uno nuevo.

Pruebas Realizadas

Corrí la verificación de tipos de TypeScript, que pasó sin errores, y recorrí los flujos completos con las cuentas de demostración. La Tabla 7 resume los casos.

Tabla 7

Casos de prueba funcionales del avance

#	Caso	Resultado esperado	Resultado
1	Login con correo mal escrito	“Correo inválido” bajo el campo, borde rojo	Correcto
2	Login con contraseña incorrecta	“Correo o contraseña incorrectos”	Correcto
3	Login de clienta y de administradora	Cada rol ve sus propias pestañas	Correcto
4	Registro con datos inválidos	Un mensaje por cada campo incorrecto	Correcto
5	Registro con código CAMI2026	Cuenta creada; la referente ve una referida más	Correcto
6	Elegir 3 servicios (3 h)	Total “3h · $73.00”; horarios de 17:00 en adelante tachados	Correcto
7	Confirmar sin aceptar la política	Mensaje de error, la cita no se crea	Correcto
8	Confirmar cita válida	Comprobante y la cita aparece en “Próximas”	Correcto
9	Cancelar cita con más de 12 h	Pide confirmación y pasa a historial como cancelada	Correcto
10	Clienta con retoque vencido	Aviso en inicio y en historial	Correcto
11	Administradora: confirmar, iniciar y completar	Solo se ofrecen transiciones válidas; completar cierra la cita	Correcto
12	Cerrar y volver a abrir la app con sesión activa	La sesión se restaura sin pedir credenciales	Correcto

Nota. Elaboración propia. Pruebas ejecutadas el 22 de octubre de 2026 con los datos de demostración.

Conclusiones y Recomendaciones

Conclusiones

Diseñar en Figma con variables y Auto Layout hizo que la implementación fuera casi una traducción. Cada propiedad de Auto Layout tiene su equivalente en Flexbox, y los nombres de las variables pasaron tal cual al archivo de tokens. Casi no perdí tiempo midiendo pantallas, y ese tiempo lo dediqué a la lógica de negocio.

Compartir lógica entre plataformas funcionó. Los módulos de disponibilidad, estados de cita e inventario de la web corren sin cambios en el móvil. Eso se logró porque en la Etapa 2 los escribí como funciones puras, aparte de Next.js y de la base de datos; si los hubiera mezclado con los endpoints, habría tenido que reescribirlos.

Las decisiones de interfaz que más pesaron tienen que ver con el diagnóstico más que con la estética. Los horarios tachados atacan las citas encimadas. Aceptar la política de cancelación de forma explícita apunta a las inasistencias. Y el aviso de retoque busca que las clientas vuelvan más seguido.

También encontré errores del prototipo que no vi hasta implementarlo, como el poco contraste del botón del login. Por eso creo que revisar el diseño contra un criterio objetivo como WCAG tiene que hacerse durante la implementación, como parte del diseño, y no dejarse para el final.

Recomendaciones

- Validar la aplicación con la propietaria y con dos o tres clientas habituales en sus propios teléfonos mediante Expo Go antes del Foro 2.
- En el Foro 2, reemplazar únicamente los servicios de la capa de datos por Firebase Authentication y Cloud Firestore, y escribir las reglas de seguridad junto con cada colección.
- Llevar a Figma los frames de registro y de la vista de la administradora para que el prototipo vuelva a reflejar la aplicación completa.

- Mover las imágenes de la galería a un servicio gratuito de alojamiento, como Cloudinary, para que la propietaria publique trabajos sin publicar una nueva versión de la app.

Referencias

Expo. (s.f.). *Expo documentation*. https://docs.expo.dev/

Figma. (s.f.-a). *Guide to auto layout*. Figma Learn. https://help.figma.com/hc/en-us/articles/360040451373

Figma. (s.f.-b). *Guide to variables in Figma*. Figma Learn. https://help.figma.com/hc/en-us/articles/15339657135383

Figma. (s.f.-c). *Guide to Dev Mode*. Figma Learn. https://help.figma.com/hc/en-us/articles/15023124644247

García Villeda, E. A. (2026). *Black Cat Nails Web: Análisis y planificación de una aplicación multiplataforma para la gestión de un emprendimiento de servicios de manicura* [Trabajo de cátedra no publicado]. Universidad Don Bosco.

McDonnell, C. (s.f.). *Zod: TypeScript-first schema validation*. https://zod.dev/

Meta Platforms. (s.f.-a). *React: The library for web and native user interfaces*. https://react.dev/

Meta Platforms. (s.f.-b). *Core components and native components*. React Native. https://reactnative.dev/docs/intro-react-native-components

Meta Platforms. (s.f.-c). *About the New Architecture*. React Native. https://reactnative.dev/architecture/landing-page

Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

React Navigation. (s.f.). *React Navigation documentation*. https://reactnavigation.org/docs/getting-started

Román Gallardo, A., Flores Cortés, C. A., & Herrera Morales, J. R. (s.f.). *Programación de aplicaciones móviles con React Native*. Editorial BUK. https://cbues.bibliotecasdigitales.com/

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Apéndice

Enlaces del Avance

Repositorio: https://github.com/Kikeagv/black-cat-nails-mobile (rama `enrique-garcia`)

Prototipo en Figma: https://www.figma.com/design/HuzR5fGLVFFeSiOJksZZls/CN?node-id=92-2

Versión web (Etapa 2): https://black-cat-nails-web.vercel.app

Video de demostración: [pegar aquí el enlace del video]

Cuentas de Demostración

Clienta: camila@mail.com · Clienta con retoque vencido: sofia@mail.com · Administradora: vante@blackcatnails.sv. Contraseña de las tres cuentas: Password123.